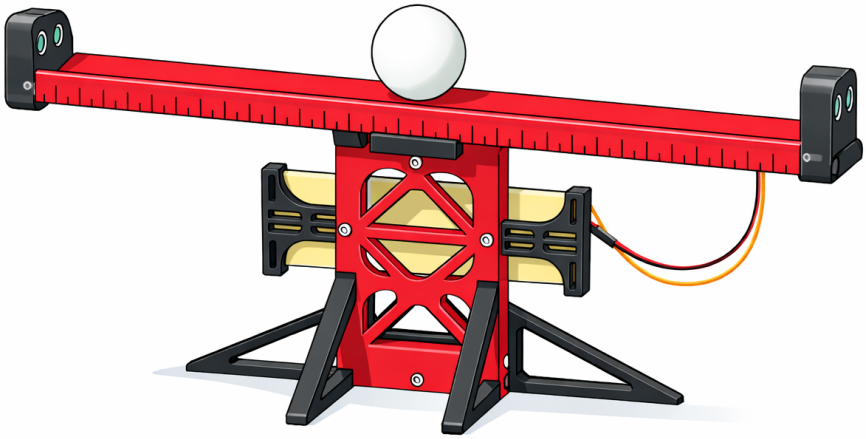
CE2LAB

Ball and beam laboratory

User handbook



Organization:
UNIVERSIDAD DE ALMERÍA
TEP197 ARM Automatic Control, Robotics and Mechatronics
IFAC INTERNATIONAL FEDERATION OF AUTOMATIC CONTROL

Contents

1 Introduction **3**

2 Design, development, and programming of the Ball and Beam **4**
- 2.1 Summary of Mechanical Construction . . . 4
- 2.2 Design . . . 4
- 2.3 Development . . . 5
  - 2.3.1 Part printing . . . 5
  - 2.3.2 Assembly of parts . . . 5
  - 2.3.3 Electronic assembly . . . 6
  - 2.3.4 Bill of Materials . . . 7
- 2.4 Programming . . . 7
- 2.5 Web-Based Interface for Interactive Tuning . . . 8

3 Background **11**
- 3.1 Modeling from First Principles . . . 11
  - 3.1.1 Assumptions and Coordinates . . . 11
  - 3.1.2 Ball Dynamics on the Beam (from Newton/Lagrange) . . . 11
  - 3.1.3 Actuator Dynamics (Motor/Servo + Geartrain) . . . 13
  - 3.1.4 Beam Angle Mapping (Servo Output to Beam Tilt) . . . 14
  - 3.1.5 Overall Linear Model: Voltage → Ball Position . . . 14
  - 3.1.6 State-Space Form . . . 15

4 Pre-Lab Questions **16**

5 Control Experiments **19**
- 5.1 Experiment 1: Inner-Loop Motor Position Control . . . 19
  - 5.1.1 Step 1 — Actuator model . . . 20
  - 5.1.2 Step 2 — Choose time-domain specifications for the inner loop . . . 20
  - 5.1.3 Step 3 — PD controller structure . . . 20
  - 5.1.4 Step 4 — Simulink implementation and validation . . . 21
- 5.2 Experiment 2: Outer-Loop Ball Position Control (Cascade) . . . 22
  - 5.2.1 Step 1 — Cascade design principle . . . 22
  - 5.2.2 Step 2 — Time-domain specifications for ball position . . . 22
  - 5.2.3 Step 3 — PD controller for ball position (outer loop) . . . 22
  - 5.2.4 Step 4 — Simulink implementation and validation (linear vs. nonlinear) . . . 23

6 Lab Report **25**
- 6.1 Report checklist for this chapter . . . 25
- 6.2 Template for Content (Cascade Control with Ideal PD Experiments) . . . 25
  - 6.2.1 Introduction . . . 25
  - 6.2.2 System Model Summary . . . 26

6.2.3 Inner-Loop Controller Design . . . 26
6.2.4 Inner-Loop Simulation Results . . . 27
6.2.5 Outer-Loop Controller Design . . . 27
6.2.6 Full Cascade Simulation Results . . . 28
6.2.7 Discussion . . . 28
6.3 Template for Content (Cascade Control with Practical PD Experiments) . . . 29
6.3.1 Overview and Objectives . . . 29
6.3.2 Experimental Results . . . 29
6.3.3 Comparison: Hardware vs. Simulation . . . 30
6.3.4 Discussion . . . 30
6.4 Tips for Report Format . . . 31
6.4.1 Recommended Structure . . . 31
6.4.2 Guidelines for Figures . . . 31
6.4.3 Equation and Notation Conventions . . . 32
6.4.4 Guidelines for Tables . . . 32
6.4.5 Common Mistakes to Avoid . . . 32
6.4.6 Indicative Grading Rubric . . . 33

1 Introduction

This laboratory workbook has been developed within the framework of the project promoted by the IFAC Technical Committee 9.4 on Control Education, the IFAC Subcommittee on Benchmark Control Problems, and the IFAC Education Activities Committee, under the CE2LAB initiative.

The CE2LAB project aims to foster the development of low-cost, accessible, and pedagogically effective laboratory platforms for control education at different academic levels. The main objective is to provide students and educators with benchmark systems that allow the practical implementation of modeling, simulation, and control design techniques in a structured and reproducible manner.

In this context, the Ball and Beam system is a classic benchmark problem in control engineering. It is widely used to illustrate key concepts such as nonlinear modeling, linearization, cascade control design, proportional-derivative (PD) control, and practical implementation issues, including actuator dynamics and sensor limitations.

This workbook is designed primarily for undergraduate and advanced-level students in engineering programs. It guides students progressively through fundamental experiments, starting with first-principles modeling and advancing to cascade control implementation and experimental validation. The material is structured to support both simulation-based learning and real-time implementation.

The modeling and control strategies presented in this workbook are implemented using MATLAB and SIMULINK (MathWorks), providing students with hands-on experience using widely adopted industrial and academic tools. In addition, the document includes guidance regarding the hardware components required to construct the Ball and Beam system. The design files, assembly instructions, and supplementary materials are available through the project GitHub repository (link to be updated).

In addition to the design files and the control firmware, the project provides a browser-based interface that allows the PID gains to be modified while the system is running, without recompiling the firmware or installing any software. This interface is distributed under the same open licence as the rest of the platform.

The pedagogical structure and several experimental guidelines are inspired by the Quanser Ball and Beam Workbook Manual, which has long served as a reference framework for laboratory-based control education. The present document adapts and extends that structure within the CE2LAB philosophy, emphasizing affordability, reproducibility, and openness.

Through this laboratory experience, students are expected to strengthen their understanding of dynamic systems modeling, controller design, performance evaluation, and practical implementation challenges in real-world control systems.

2 Design, development, and programming of the Ball and Beam

2.1 Summary of Mechanical Construction

The Ball and Beam system presented in this paper has been developed by the Automation, Robotics and Mechatronics (ARM) group at the University of Almería and is available on the GitHub repository *Ce2Lab_ball_and_beam*[1]. This design constitutes a low-cost, self-assembly tool, allowing the exploration of control theory principles through both hardware and software implementations. The mechanical architecture consists of a centrally supported model, whose components have been manufactured entirely using 3D printing. Following the assembly process, the sensors and actuators are integrated with the central processing unit to run the control firmware, resulting in the final model. The following sections detail the technical characteristics of each subsystem.

2.2 Design

The design process was based on the Arduino PID Balance Beam project[2]. However, significant structural modifications were made, as the original model had functional and manufacturing limitations. The optimizations to the additive manufacturing components are detailed below:

- `Main_Body.stl`: This was redesigned to optimise material usage and adapted to a dual-support configuration, incorporating a shaft for securing the running rails.
- `Pivot_.stl`: The geometry was modified to transition from a single pivot point to a dual-support system on the main body.
- `Servo_Base_Mount.stl`: Completely redesigned to allow the transition from the original SG90 servo motor to the MG996R model, in order to increase the system's torque.
- `Sensor_Rail_End.stl`: This component was discarded following the replacement of the HC-SR04 ultrasonic sensor with a laser sensor (VL53L0X).
- `Rail_End.stl`: A new variant was developed, specifically adapted for the mounting and alignment of the VL53L0X sensor.
- `Rail_with_Ruler.stl` and `Rail.stl`: The design was broken down into modular sections to facilitate manufacturing, overcoming the print volume limitations of conventional equipment.

These modifications resulted in the final design of the Ball and Beam system proposed in this research.

[1] Oficial project repository: https://github.com/FerCanAra/Ce2Lab_ball_and_beam
[2] Model on Thingiverse: https://www.thingiverse.com/thing:6387659

2.3 Development

2.3.1 Part printing

Once all the designs were complete, the parts were printed using Prusa's iPrusa Slicer software, with the following settings:

- Print settings:
  - Layer height: 0.15 mm
  - First layer height: 0.2 mm
  - Infill: 5 %
  - Raft: No
  - Supports: Yes
  - Ironing: 15 mm/s
- Material parameters:
  - PLA 1.75 mm
  - Extruder temperature: 215 °C
  - Bed temperature: 60 °C

The process will take 34 hours and 35 minutes, using a total of 341.53 g

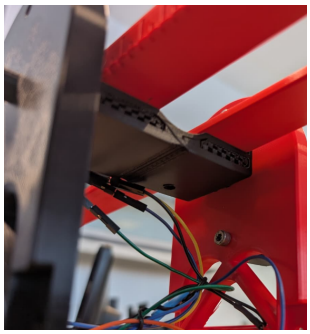
2.3.2 Assembly of parts

Once the components had been manufactured, the system was assembled using a step-by-step approach. The mechanical integration process is described below:

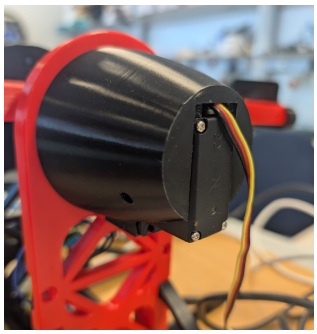
1. The side supports `Base_Support_Leg_Right.stl` were attached to the main body `Main_Body.stl` using M4 hexagonal (Allen) screws. Structural integrity between these elements was ensured by the intermediate component `Arduino_Board_Base.stl` (Figure 2.1a).
2. The MG966R servo motor was integrated into the mount `Servo_Base_Mount.stl`, which was subsequently attached to the central chassis (`Main_Body.stl`) (Figure 2.1b).
3. The running components, `Rail_End.stl` and `Rail_with_Ruler.stl`, were fixed to the pivoting element (`Pivot.stl`) using a high-viscosity thermoplastic adhesive to ensure alignment.
4. The rail ends were joined by integrating the `Sensor_Rail_End.stl` parts into the ends of the rail structure.
5. The VL53L0X ToF sensor was installed and secured in its specific housing (`Sensor_Rail_End.stl`). At the same time, the control unit based on the ATMega328P microcontroller was mounted on the `Arduino_Board_Base.stl` base (Figure 2.1c).

6. Finally, the electrical interconnection of the transducers and actuators was carried out in accordance with the diagrams detailed in section 2.3.3.

The result of the assembly is shown in Figure 2.2.



(a) Body assembly



(b) Motor assembly



(c) Sensor assembly

Figure 2.1: Assembly part

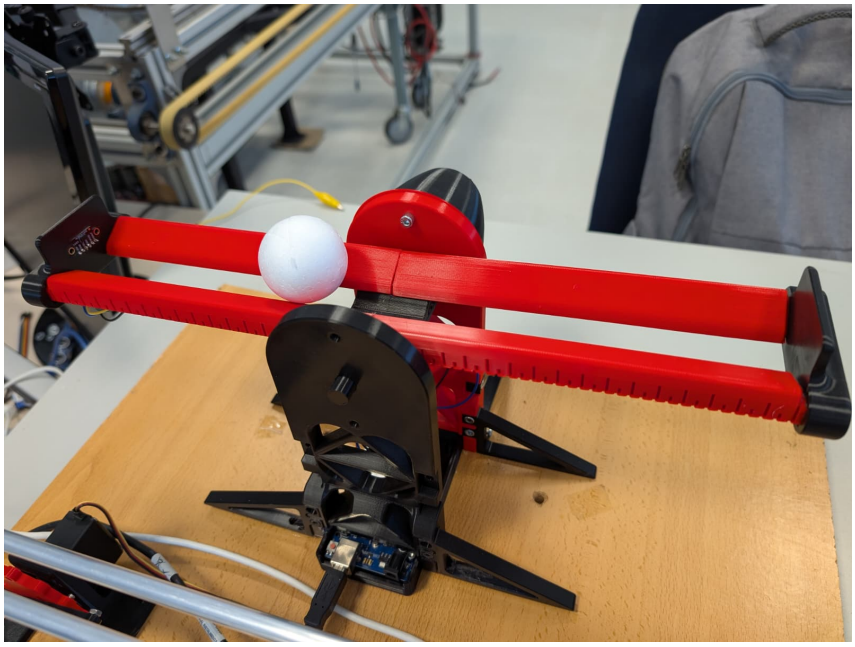


Figure 2.2: Final model

2.3.3 Electronic assembly

The system's electronic integration is based on a communication architecture. For the sensor interface, the analog pins `A1` (SCL) and `A2` (SDA) are used, establishing communication via the I^2C protocol. The actuator is controlled via digital pin `11`, configured to transmit control signals to the servomotor. Power distribution is provided via common 5 V power buses and ground (GND). Physical interconnection is achieved using jumper-type wires compatible with the development board's pin spacing. A detailed diagram of this connection is shown in Figure 2.3.

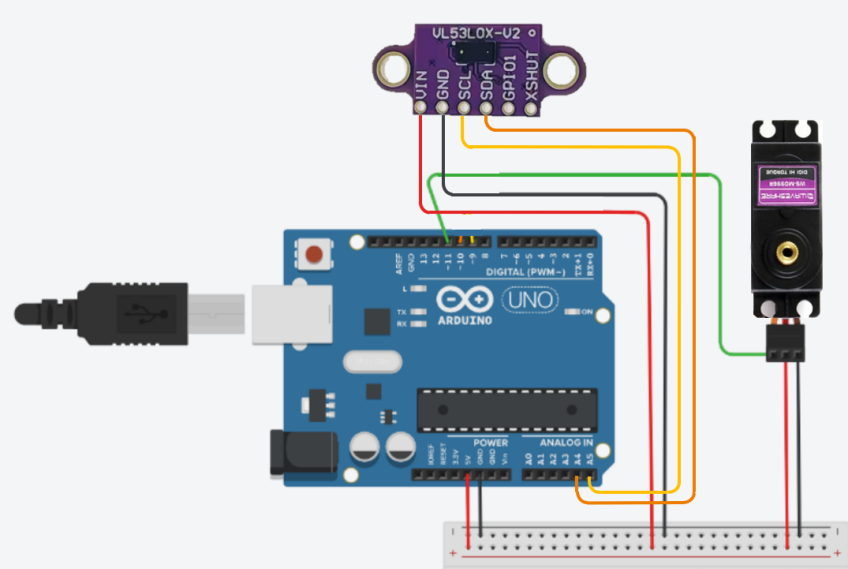


Figure 2.3: Electronic assembly

2.3.4 Bill of Materials

The following parts list includes the main components required to assemble and operate the Ball and Beam system. The list below contains all the essential mechanical, electrical, and structural components necessary to construct a fully functional laboratory setup.

Table 2.1: Bill of Materials for the Ball and Beam System

Part	**Qty.**	**Price (€)**	**Notes**
M4 Allen scrws	12	1	Screws for securing parts
Jumper wire MF	7	0.5	Electrical connection cables
Hot glue	1	0.2	Hot glue for joining pieces
Ping-pong Ball	1	0.5	An object that will slide along the guides
Micontroller ATMega328P	1	10	Controller responsible for the control loop
MG996r	1	6	5V servo motor
PLA roll (350 g)	1	6.5	Polylactic acid for 3D printing
VL53L0X	1	1	Distance sensor
Total cost		**25.20 €**	

2.4 Programming

The software development was initially based on the control structure proposed in Ball-Beam-PID-Control[3] repository, although substantial modifications were implemented to optimise the system's performance. The integrated development environment (IDE) selected was `PlatformIO` within the `Visual Studio Code` (VSC) ecosystem, utilising its advanced dependency manager for the integration of the `VL53L0X` sensor control libraries, servo motor management and the PID algorithm. It should be noted that the PID control library was extensively modified; consequently, the optimised version of the algorithm has been made available to the community in

[3]Base code template: https://github.com/AntonAshraf/Ball-Beam-PID-Control/tree/main

the project's official repository.

The software is deployed onto the ATmega328P microcontroller via a serial communication interface. The workflow consists of identifying the communication port, compiling the binary and subsequently uploading it to the development board. Once this process is complete, the Ball and Beam system is ready for the validation phase. The full source code and configuration files are available in the official repository mentioned above.

2.5 Web-Based Interface for Interactive Tuning

The platform includes a browser-based interface intended both for laboratory sessions and for public demonstrations. The design follows a deliberate separation of concerns: the complete control loop—sensor acquisition, filtering, PID computation and servo actuation—runs on the ATMega328P at approximately 30 Hz, while the computer is used only to modify parameters and to display the incoming data. Communication uses a plain-text serial protocol at 115 200 baud, with one command per line. As a consequence, the platform remains fully operational if the computer is disconnected or the browser is closed, a property that matters when the system is left running unattended during a demonstration or shared among several student groups.

The interface is a single self-contained HTML file that accesses the serial port through the Web Serial API, available in Chromium-based browsers. It requires no installation, no runtime, no commercial licence and no network connection, so it can be copied to any laptop and opened directly. This keeps the software side of the platform aligned with the cost and accessibility objectives of the hardware design.

Figure 2.4 shows the interface. The upper area renders the beam and the ball in real time, together with the current position, target, error and beam angle. The target can be modified by dragging on the beam itself or with a slider. The three gain sliders are accompanied by a one-line description of the role of each term and of the symptom produced when it is set too high.

The central pedagogical element is the panel labelled *Who is tilting the beam right now?*, which decomposes the commanded beam angle into the individual contributions of the proportional, integral and derivative terms and displays them as signed bars updated at the sampling rate. The same decomposition is plotted over a 20-second window below the position history. Students are therefore able to observe not only that the response changed, but which term produced the change—a relationship that is usually presented algebraically and is considerably harder to convey from a single closed-loop response plot.

A set of predefined gain combinations makes the qualitative behaviours discussed in Section 3 immediately reproducible: proportional action alone, which sustains an oscillation because the plant is a double integrator and nothing opposes the ball velocity; proportional–derivative action, which stabilises the response but leaves a steady-state offset when the beam is not perfectly level; the complete PID, which removes that offset; and deliberately detuned cases with excessive integral action or insufficient damping. Each selection is accompanied by a short explanatory message.

Two implementation aspects of the firmware are relevant to the quality of the experimental data obtained through the interface. First, the raw output of the time-of-flight sensor exhibits a noise level of 20–30 mm, so numerical differentiation of the measurement produces a control

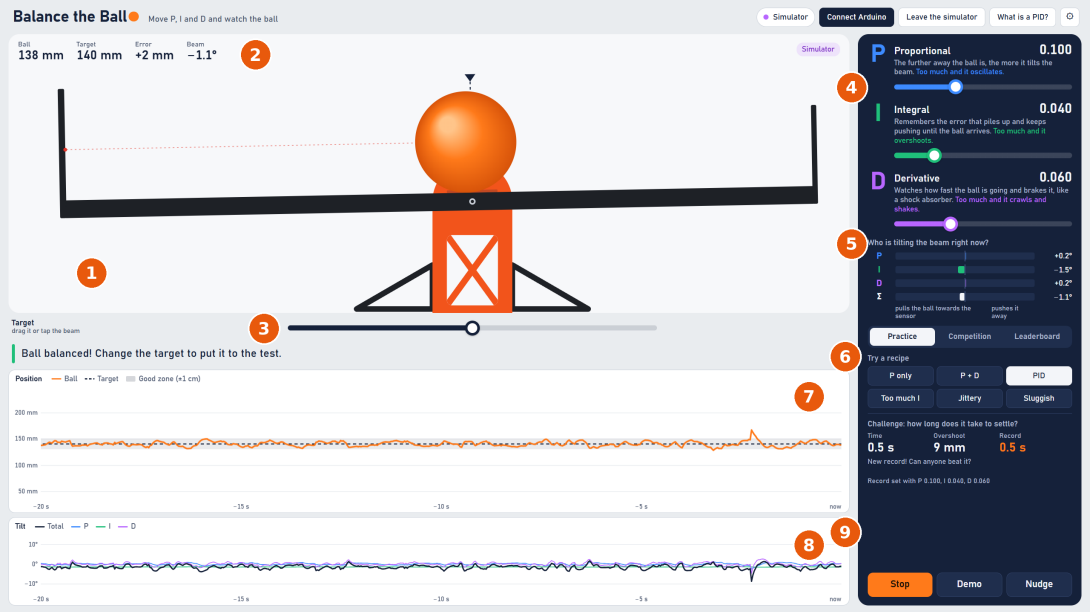


Figure 2.4: Web-based interface of the Ball and Beam platform, shown here running against the embedded simulator: (1) real-time rendering of the beam and the ball, where the target can be set by dragging; (2) instantaneous values of ball position, target, error and beam angle; (3) target slider; (4) PID gain sliders with a plain-language description of each term; (5) decomposition of the commanded beam angle into the P, I and D contributions; (6) predefined gain sets reproducing the characteristic behaviours of Section 3; (7) ball position history with the target and a ± 10 mm tolerance band; (8) commanded tilt history decomposed into its three terms; (9) settling-time challenge, together with the competition and leaderboard tabs used in outreach activities.

signal dominated by noise. A median-of-three filter followed by an alpha–beta observer is used instead, in which the prediction step uses the identified plant model, $\ddot{x} = K_b(\alpha + \alpha_0)$, and a slow third gain estimates the residual offset α_0 caused by imperfect levelling of the beam. In simulation, this reduces the mean sample-to-sample variation of the commanded servo angle from approximately 2.6° with direct differentiation to approximately 0.6°. The residual-offset estimate also prevents the estimated velocity from being biased when the beam is not exactly horizontal, which would otherwise be attributed incorrectly to the derivative action during the laboratory exercises. Near the mechanical end stops the model prediction is disabled and the estimate of α_0 is frozen, since the reaction of the end stop is not represented in the model.

Second, the derivative term acts on the measurement rather than on the error, so that a change of target does not produce an impulsive command, and the integral term uses conditional anti-windup: accumulation is suspended whenever the command is already saturated in the direction of the error. Both choices are standard in industrial implementations and are visible to the student in the decomposition panel, which provides a convenient opportunity to discuss the difference between the textbook PID and its practical form.

The interface also embeds a simulator that executes the same controller code against the identified model of the platform, including the measurement noise, the servo time constant and a configurable levelling offset. The simulator allows the tuning exercises to be completed without the physical system, which is useful for preparation before the laboratory session, for a larger cohort sharing a single rig, or for institutions that have not yet built the hardware. It also makes

the qualitative difference between the linear and nonlinear predictions of Section **??** directly observable by the student.

The serial protocol is documented in Table 2.2 so that the firmware can be driven from any other environment. The MATLAB/Simulink interface used in the experiments of Section 6.3 and the browser interface described here are two clients of the same protocol, and users may implement their own in Python, LabVIEW or any language with serial port access without modifying the firmware.

Table 2.2: Serial protocol of the Ball and Beam firmware (115 200 baud, one command per line).

Command	**Meaning**
Computer → Arduino	
`P<v> I<v> D<v>`	PID gains, in °/mm, °/(mm·s) and °·s/mm
`S<v>`	Ball position setpoint (mm)
`M<n>`	Operating mode: 0 stopped (beam level), 1 PID, 2 manual
`A<v>`	Commanded beam angle in manual mode (°)
`L<v>`	Beam angle saturation limit (°)
`N<v>`	Servo angle corresponding to a level beam (°)
`X<n>`	Servo direction, $+1$ or -1
`R W ?`	Reset integral term, store settings in EEPROM, request configuration
Arduino → computer	
`$t,pos,sp,u,p,i,d, mode,raw,ok,offset`	Telemetry frame, ≈30 Hz
`#CFG,...`	Current configuration, sent on connection
`#SAVED #ERR,SENSOR #READY`	Status messages

3 Background

3.1 Modeling from First Principles

This section derives a physics-based model for the Ball and Beam system, separating the dynamics into: (i) the **ball motion on an inclined beam**, and (ii) the **actuator (motor/servo) dynamics** that generate the beam angle. The motor model follows the standard DC servo/geartrain derivation used in Quanser educational material and is consistent with the equations summarized in the provided motor modeling document.

3.1.1 Assumptions and Coordinates

We model a single-axis Ball and Beam setup (one degree of freedom along the beam). The main assumptions are:

- The ball rolls without slipping on the beam (pure rolling).
- Frictional losses are neglected in the ball dynamics (dominant effects captured by rolling constraint).
- Beam angles remain small during normal operation: $\sin\alpha \approx \alpha$, $\cos\alpha \approx 1$.
- The beam is rigid, and its flexural dynamics are neglected.

Let:

- $x(t)$: ball position along the beam (measured from the pivot/beam origin).
- $\alpha(t)$: beam angle (rad), positive when the beam is tilted so gravity accelerates the ball in the $+x$ direction.
- r_b: ball radius, m_b: ball mass, J_b: ball moment of inertia about its center.

For a solid sphere, $J_b = \frac{2}{5} m_b r_b^2$.

3.1.2 Ball Dynamics on the Beam (from Newton/Lagrange)

The mean scheme of the Ball and Beam system is represented in Fig.3.1

Along the beam direction, the gravitational component is $m_b g \sin\alpha$. The ball both translates and rotates. With rolling without slipping,

$$\dot{x}(t) = r_b\,\omega(t) \quad \Rightarrow \quad \ddot{x}(t) = r_b\,\dot{\omega}(t). \tag{3.1}$$

Force balance along the beam:

$$m_b \ddot{x}(t) = m_b g \sin\alpha(t) - F_f(t), \tag{3.2}$$

where F_f is the (static) friction force that enforces rolling.

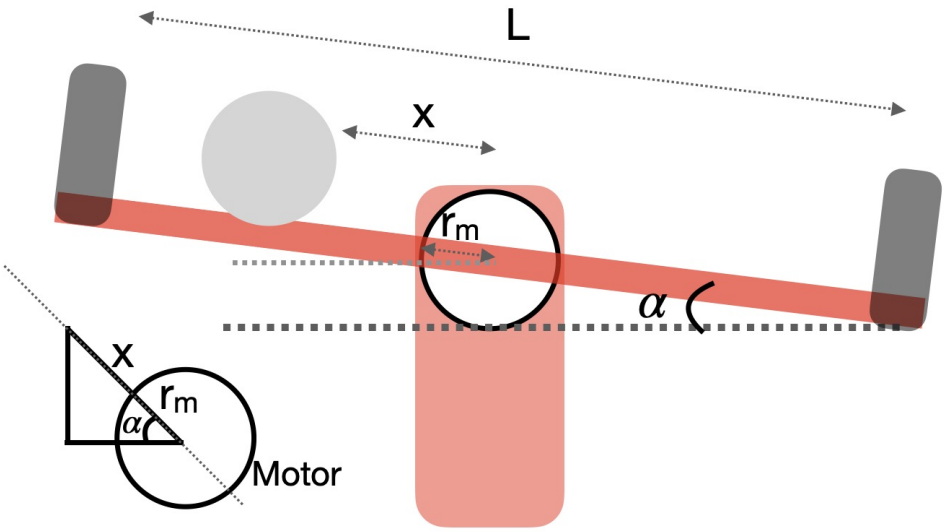


Figure 3.1: Ball and beam scheme

Torque balance about the ball center:

$$J_b\dot{\omega}(t) = F_f(t)r_b. \tag{3.3}$$

Eliminate $F_f(t)$ using $\dot{\omega}(t) = \ddot{x}(t)/r_b$:

$$J_b\frac{\ddot{x}(t)}{r_b} = F_f(t)r_b \quad \Rightarrow \quad F_f(t) = \frac{J_b}{r_b^2}\ddot{x}(t). \tag{3.4}$$

Substitute into the translational equation:

$$m_b\ddot{x}(t) = m_b g\sin\alpha(t) - \frac{J_b}{r_b^2}\ddot{x}(t) \tag{3.5}$$

$$\left(m_b + \frac{J_b}{r_b^2}\right)\ddot{x}(t) = m_b g\sin\alpha(t). \tag{3.6}$$

Therefore, the **nonlinear** ball dynamics are:

$$\ddot{x}(t) = \frac{m_b g}{m_b + \frac{J_b}{r_b^2}}\sin\alpha(t). \tag{3.7}$$

Using the solid-sphere inertia $J_b = \frac{2}{5}m_b r_b^2$, we get the common compact form:

$$\ddot{x} = \frac{g}{1+\frac{2}{5}}\sin\alpha = \frac{5}{7}g\sin\alpha. \tag{3.8}$$

Small-angle Linearization

For small angles, $\sin\alpha(t) \approx \alpha(t)$, hence:

$$\ddot{x}(t) \approx k_b\alpha(t), \tag{3.9}$$

for $k_b := \frac{m_b g}{m_b + \frac{J_b}{r_b^2}}$ $\left(\text{for a solid sphere: } k_b = \frac{5}{7} g\right)$.

Taking Laplace transforms (zero initial conditions):

$$P_b(s) = \frac{X(s)}{\alpha(s)} = \frac{k_b}{s^2}. \tag{3.10}$$

3.1.3 Actuator Dynamics (Motor/Servo + Geartrain)

The beam angle $\alpha(t)$ is produced by a motor/servo mechanism. Although the actuator is a stepper motor, for control design purposes, it is modeled as an equivalent servo actuator with continuous angular dynamics. This approximation is valid during microstepping and within the experiment's normal operating region.

In many educational setups, a DC motor (with a gearbox) drives an output shaft whose rotation (or a connected linkage) sets the beam angle. The motor model below follows the standard electrical and mechanical derivation provided in the motor modeling PDF.

Electrical Equation

For a DC motor, the armature voltage is:

$$V_c(t) = R_c i_c(t) + L_c \frac{di_c(t)}{dt} + e_b(t), \tag{3.11}$$

where R_c is armature resistance, L_c inductance, i_c current, and e_b back-emf. The back-emf is proportional to motor speed:

$$e_b(t) = k_e \omega_m(t). \tag{3.12}$$

In many servo motors, $L_c \ll R_c$ and is neglected, yielding:

$$V_c(t) = R_c i_c(t) + k_e \omega_m(t). \tag{3.13}$$

Mechanical Equation with Geartrain

Motor torque is proportional to current:

$$\tau_m(t) = \eta_m k_t \, i_c(t), \tag{3.14}$$

where k_t is the torque constant and η_m is motor efficiency.

Let $\theta(t)$ be the **load/output** shaft angle (after gearbox), with speed $\omega(t) = \dot{\theta}(t)$. A gear ratio K_g relates motor and load speeds:

$$\omega_m(t) = K_g \, \omega(t). \tag{3.15}$$

The motor/load inertias and viscous frictions can be combined into equivalent parameters at the load side:

$$J_{eq} \, \dot{\omega}(t) + B_{eq} \, \omega(t) = \eta_g K_g \, \tau_m(t), \tag{3.16}$$

where η_g is gearbox efficiency, J_{eq} is the equivalent inertia at the load shaft, and B_{eq} the equivalent viscous damping.

From Voltage to Load Angle

Combine (3.13)–(3.16). From (3.13):

$$i_c(t) = \frac{1}{R_c}\left(V_c(t) - k_e\omega_m(t)\right) = \frac{1}{R_c}\left(V_c(t) - k_e K_g \omega(t)\right). \tag{3.17}$$

Substitute into (3.14) and then into (3.16):

$$J_{eq}\dot{\omega}(t) + B_{eq}\omega(t) = \eta_g K_g \eta_m k_t \frac{1}{R_c}\left(V_c(t) - k_e K_g \omega(t)\right). \tag{3.18}$$

Rearrange:

$$J_{eq}\dot{\omega}(t) + \underbrace{\left(B_{eq} + \frac{\eta_g \eta_m k_t k_e K_g^2}{R_c}\right)}_{B_{eq}^\star}\omega(t) = \underbrace{\left(\frac{\eta_g \eta_m k_t K_g}{R_c}\right)}_{G_m} V_c(t). \tag{3.19}$$

Since $\omega(t) = \dot{\theta}(t)$, the motor/servo dynamics can be written as:

$$J_{eq}\ddot{\theta}(t) + B_{eq}^\star \dot{\theta}(t) = G_m\, V_c(t). \tag{3.20}$$

In Laplace form:

$$P_m(s) = \frac{\Theta(s)}{V_c(s)} = \frac{G_m}{s\left(J_{eq}s + B_{eq}^\star\right)} = \frac{K_\theta}{s(\tau_m s + 1)}, \tag{3.21}$$

where $K_\theta = \frac{G_m}{B_{eq}^\star}$ and $\tau_m = \frac{J_{eq}}{B_{eq}^\star}$.

3.1.4 Beam Angle Mapping (Servo Output to Beam Tilt)

Since the motor shaft is aligned with the beam rotation axis (direct drive), the beam angle coincides with the motor/load angle:

$$\alpha(t) = \theta(t). \tag{3.22}$$

Thus:

$$P_m(s) = \frac{\alpha(s)}{V_c(s)} = \frac{\Theta(s)}{V_c(s)} = \frac{K_\theta}{s(\tau_m s + 1)}. \tag{3.23}$$

3.1.5 Overall Linear Model: Voltage → Ball Position

Using (3.10) and (3.23), the overall plant is:

$$\frac{X(s)}{V_c(s)} = \frac{X(s)}{\alpha(s)}\frac{\alpha(s)}{V_c(s)} = \frac{k_b}{s^2}\cdot\frac{K_\theta}{s(\tau_m s + 1)} = \frac{K}{s^3(\tau_m s + 1)}, \tag{3.24}$$

with $K := k_b K_\theta$.

3.1.6 State-Space Form

A convenient state choice is:

$$x_1(t) = x(t), \quad x_2(t) = \dot{x}(t), \quad x_3(t) = \theta(t), \quad x_4(t) = \dot{\theta}(t).$$

Using (3.9), and (3.20):

$$\dot{x}_1(t) = x_2(t), \tag{3.25}$$

$$\dot{x}_2(t) = k_b\,\alpha(t) = k_b\,x_3(t), \tag{3.26}$$

$$\dot{x}_3(t) = x_4(t), \tag{3.27}$$

$$\dot{x}_4(t) = -\frac{B^\star_{eq}}{J_{eq}}x_4(t) + \frac{G_m}{J_{eq}}V_c(t). \tag{3.28}$$

In matrix form:

$$\dot{\mathbf{x}}(t) = \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & k_b & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & -\frac{B^\star_{eq}}{J_{eq}} \end{bmatrix}}_{A}\mathbf{x}(t) + \underbrace{\begin{bmatrix} 0 \\ 0 \\ 0 \\ \frac{G_m}{J_{eq}} \end{bmatrix}}_{B}V_c(t), \qquad y(t) = \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \end{bmatrix}}_{C}\mathbf{x}(t). \tag{3.29}$$

Remarks.

- The ball subsystem contributes a double integrator $(1/s^2)$ from $\alpha(t)$ to $x(t)$.
- The motor/servo adds approximately first-order velocity dynamics plus an additional integrator from speed to position, leading to the typical $s^3(\tau s + 1)$ denominator in (3.24).
- In later sections, these models are used to motivate the cascade control structure: an inner loop regulating $\alpha(t)$ (or $\theta(t)$) and an outer loop regulating $x(t)$.

4 Pre-Lab Questions

The following pre-laboratory questions are designed to ensure that students understand the modeling, dynamics, and control principles of the Ball and Beam system before conducting the experiments. Students are expected to show all analytical steps and justify their reasoning.

Part I – Modeling and System Understanding

1. Starting from Newton's second law, derive the nonlinear equation of motion of a rolling ball on an inclined beam. Clearly state all assumptions made during the derivation.
2. Is the open-loop ball and beam system stable? Justify your answer analytically from the transfer function.
3. Explain why the small-angle approximation $\sin(\alpha) \approx \alpha$ is reasonable in this laboratory setup. What would happen if the beam angle becomes large?

Part II – Actuator Modeling

1. Consider a DC servo motor used to control the beam angle. Starting from the equation:

$$J\ddot{\theta} + B\dot{\theta} = Ku,$$

   derive the transfer function $\Theta(s)/U(s)$.
2. What is the physical meaning of the parameters J, B, and K?
3. If a servo motor includes an internal position control loop, how does this affect the effective model seen from the outer loop (ball position controller)? Would the actuator behave more like a first-order or second-order system?
4. Explain why it is common to design a cascade control structure for the Ball and Beam system (inner loop for angle, outer loop for position).

Part III – Stepper Motor Considerations

1. Unlike a DC servo motor, a stepper motor is typically driven in open loop. Explain how angular position is achieved using step pulses.
2. In the absence of an encoder, how does the system "know" the beam angle when using a stepper motor?
3. What physical phenomenon may occur if the required torque exceeds the available motor torque in a stepper motor system?

4. If a stepper motor is driven using microstepping, explain why its behavior can be approximated by a continuous-time actuator model.

5. Suppose you want to implement closed-loop beam angle control using a stepper motor. What additional hardware would be required?

6. Compare the advantages and disadvantages of using:

   - A DC servo motor with encoder feedback
   - A stepper motor without encoder

Part IV – Open-Loop Simulation and Model Validation

In this part, students must use the provided Simulink model `openloop_bb.slx`. The model contains both the linear and nonlinear versions of the Ball and Beam dynamics.

Students are required to perform simulations and analyze the behavior of the system in open loop.

1. Open the Simulink model `openloop_bb.slx`. Identify:

   - The actuator subsystem
   - The beam angle output
   - The linear ball dynamics
   - The nonlinear ball dynamics

2. Apply a positive unit step input to the actuator. Describe qualitatively and quantitatively the response of:

   - The motor/beam angle
   - The ball position

   Is the system stable in open loop? Explain your observations.

3. Repeat the experiment with larger step amplitudes. Compare the linear and nonlinear model responses.

   At what range of beam angles does the linear approximation begin to noticeably deviate from the nonlinear model?

4. Apply the following input sequence:

   - +1 from $t = 5$ to 5.5 s
   - -1.2 from $t = 5.5$ to 6 s
   - 0 from $t = 6$ s onward

   Plot and compare the ball position responses of the linear and nonlinear models. Explain any differences observed. Compare the outputs between the linear and nonlinear model to the actual system, as shown in Fig.4.1.

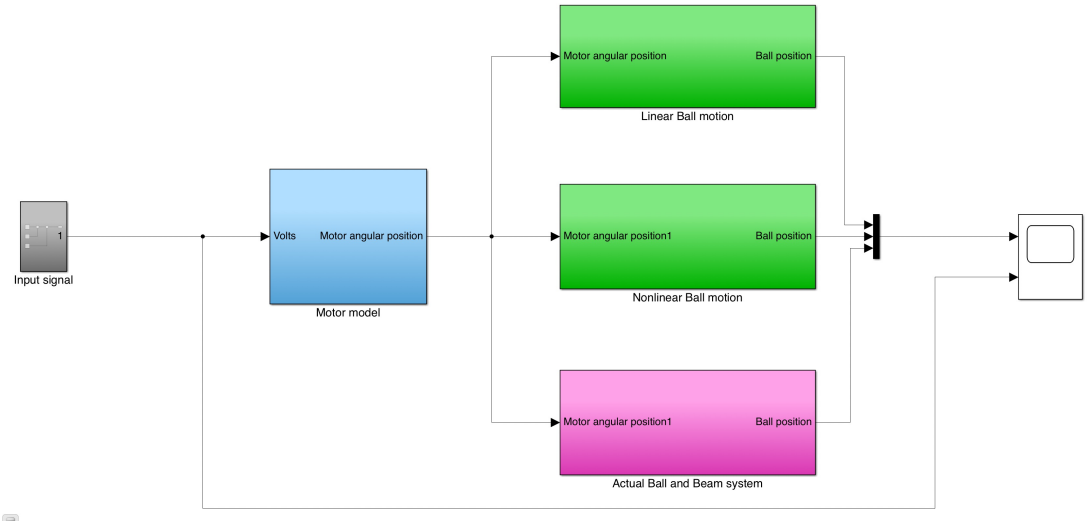


Figure 4.1: Enter Caption

5. From the motor step response (beam angle response to a voltage step), estimate the following actuator parameters:

   - The steady-state gain K_m
   - The dominant time constant τ_m

   Describe the method used to estimate these parameters from the simulation.

6. Using the ball acceleration equation:

$$\ddot{x}(t) = k_b \alpha(t),$$

   propose a method to estimate the parameter k_b from simulation data.

   (Hint: consider measuring the initial acceleration of the ball when a small constant beam angle is applied.)

7. Can the nonlinear model parameter k_b be identified independently from gravitational acceleration? Explain your reasoning.

8. If the actuator includes viscous damping B, describe how this parameter could be estimated from the motor step response.

9. Explain why certain parameters (e.g., gravity g) cannot be identified from these experiments unless additional measurements are available.

Discussion Question Based on your simulation results:

- Is the linear model sufficient for control design in this laboratory?
- Under what operating conditions would the nonlinear model be necessary?
- How does open-loop behavior justify the need for cascade closed-loop control?

5 Control Experiments

This chapter guides students through the design and validation of a **cascade control** strategy for the Ball and Beam system. The cascade structure consists of:

- an **inner loop** to control the actuator/beam angle (motor position control), and
- an **outer loop** to control the ball position along the beam.

Unless otherwise stated, use the following parameters throughout this chapter:

$$g = 9.81 \ \ [\mathrm{m/s^2}],$$
$$k_b = \frac{5}{7}g,$$
$$J = 0.021, \qquad B = 0.15, \qquad K = 5.9586.$$

Students should observe and understand the three block diagrams shown in Figures 5.1–5.3:

1. Open-loop system,
2. Inner-loop closed-loop (motor position/beam angle control),
3. Full cascade closed-loop (ball position control using an inner angle loop).

5.1 Experiment 1: Inner-Loop Motor Position Control

The objective of the inner loop is to achieve a fast and well-damped response of the actuator angle. In a classical Ball and Beam setup, the inner loop regulates either:

- the motor/load angle θ (position control), or
- the beam angle α directly (if an angle sensor is available).

Important note (servo/stepper users). If your platform already includes an internal servo controller (closed-loop position control inside the actuator) **or** if you use a stepper motor in open-loop position mode where the commanded angle is accurately achieved at the speeds used in the lab, you may treat the actuator as an *ideal angle source*. In that case, **skip to Section 5.2**.

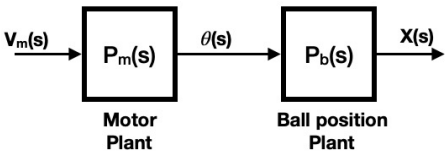


Figure 5.1: Open-loop block diagram: actuator dynamics followed by ball dynamics.

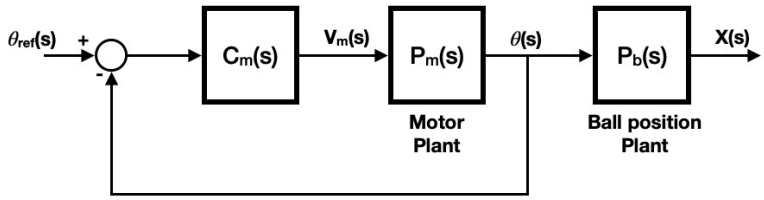


Figure 5.2: Inner-loop block diagram for motor position (or beam angle) control.

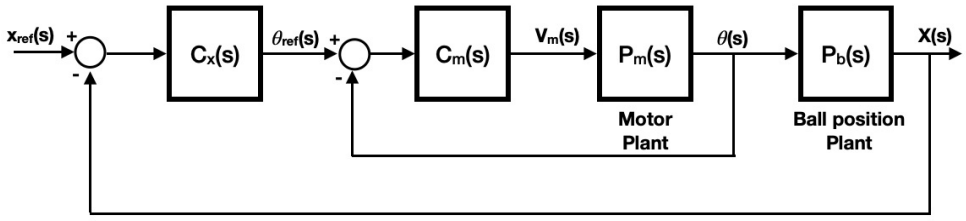


Figure 5.3: Cascade control structure: outer loop regulates ball position, inner loop regulates actuator/angle.

5.1.1 Step 1 — Actuator model

For the purpose of inner-loop design, use the simplified motor position plant:

$$P_m(s) = \frac{\Theta(s)}{V_m(s)} = \frac{K}{Js^2 + Bs} = \frac{K}{s(Js + B)}. \tag{5.1}$$

5.1.2 Step 2 — Choose time-domain specifications for the inner loop

Select a reasonably fast inner-loop response. A typical choice for laboratory hardware is:

- Overshoot: $M_p \leq 10\%$
- Peak time: $t_p \approx 0.2$–0.5 s

From overshoot, choose a damping ratio ζ_θ using the standard second-order relation:

$$M_p = e^{-\frac{\zeta_\theta \pi}{\sqrt{1-\zeta_\theta^2}}}. \tag{5.2}$$

From peak time, estimate a natural frequency:

$$t_p \approx \frac{\pi}{\omega_{n\theta}\sqrt{1-\zeta_\theta^2}}. \tag{5.3}$$

5.1.3 Step 3 — PD controller structure

Use a PD controller for motor position:

$$C_\theta(s) = K_{p\theta} + K_{d\theta} s. \tag{5.4}$$

With unity feedback (Figure 5.2), the characteristic polynomial becomes:

$$Js^2 + (B + KK_{d\theta})\, s + KK_{p\theta} = 0. \tag{5.5}$$

Match it to a desired second-order form:

$$J\left(s^2 + 2\zeta_\theta \omega_{n\theta} s + \omega_{n\theta}^2\right) = 0. \tag{5.6}$$

Thus, the PD gains are obtained by:

$$K_{d\theta} = \frac{2\zeta_\theta \omega_{n\theta} J - B}{K}, \tag{5.7}$$

$$K_{p\theta} = \frac{\omega_{n\theta}^2 J}{K}. \tag{5.8}$$

5.1.4 Step 4 — Simulink implementation and validation

1. Build the inner loop in Simulink using the plant (5.1) and PD controller (5.4).Compare the linear and nonlinear models with the actual system outputs, as shown in Fig.5.4

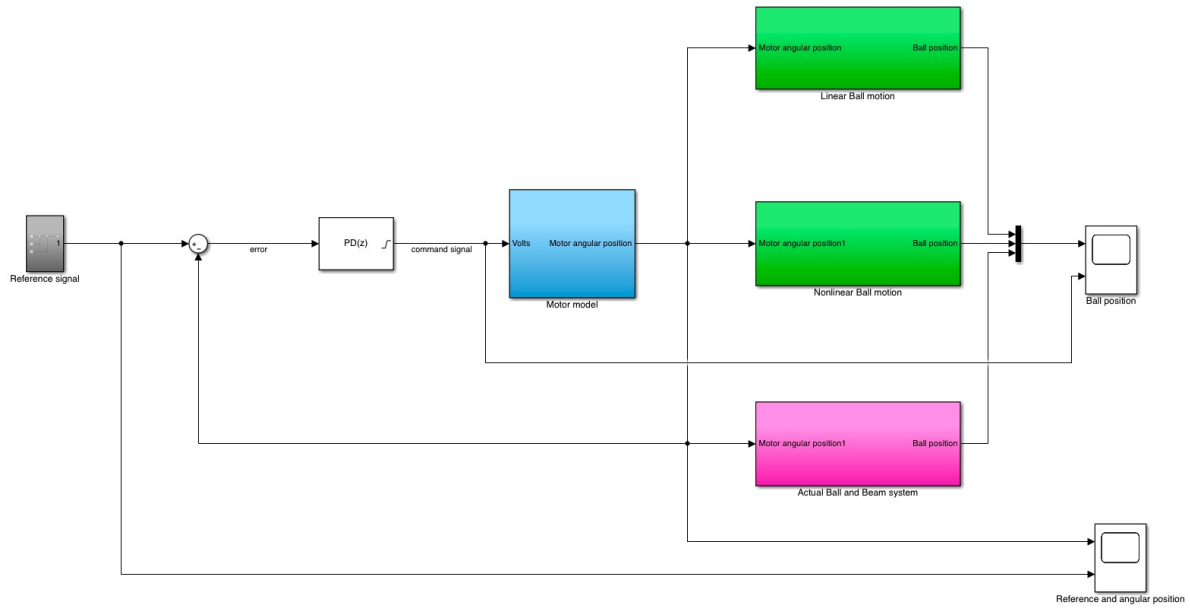


Figure 5.4: Inner loop control of the Ball and Beam motor

2. Apply a step reference θ_{ref} (small amplitude).
3. Verify:
   - Overshoot M_p,
   - Peak time t_p,
   - Steady-state tracking (note: pure PD on a type-1 plant typically yields zero steady-state error for steps).
4. If the response is too oscillatory: increase ζ_θ (increase $K_{d\theta}$).

5. If the response is too slow: increase $\omega_{n\theta}$ (increase both gains).

5.2 Experiment 2: Outer-Loop Ball Position Control (Cascade)

The outer loop regulates the ball position $x(t)$ by commanding the beam angle. Using the small-angle linearized ball model:

$$P_b(s) = \frac{X(s)}{\theta(s)} = \frac{k_b}{s^2}, \qquad k_b = \frac{5}{7}g. \tag{5.9}$$

5.2.1 Step 1 — Cascade design principle

Design the inner loop first, then the outer loop. The inner loop should be **significantly faster** than the outer loop (typical guideline: inner bandwidth 3–5× higher). Under this separation, the inner loop can be approximated as:

$$\alpha(t) \approx \alpha_{\text{cmd}}(t),$$

and the outer loop design can focus on (5.9).

5.2.2 Step 2 — Time-domain specifications for ball position

Use the following time-domain specifications for the ball position response:

- Overshoot: $M_p \leq 5\%$
- Peak time: $t_p \approx 1.5$–3 s

Select ζ_x from M_p and compute ω_{nx} from t_p:

$$t_p \approx \frac{\pi}{\omega_{nx}\sqrt{1-\zeta_x^2}}. \tag{5.10}$$

5.2.3 Step 3 — PD controller for ball position (outer loop)

Use a PD controller on the position error $e_x = x_{\text{ref}} - x$:

$$C_x(s) = K_{px} + K_{dx}s. \tag{5.11}$$

For the plant (5.9) with unity feedback on x and control signal α_{cmd}, the closed-loop characteristic polynomial becomes:

$$s^2 + k_b K_{dx} s + k_b K_{px} = 0. \tag{5.12}$$

Match it to:

$$s^2 + 2\zeta_x \omega_{nx} s + \omega_{nx}^2 = 0, \tag{5.13}$$

which yields:

$$K_{dx} = \frac{2\zeta_x \omega_{nx}}{k_b}, \tag{5.14}$$

$$K_{px} = \frac{\omega_{nx}^2}{k_b}. \tag{5.15}$$

Command mapping to the inner loop. If the inner loop regulates motor angle θ and the beam angle is $\alpha \approx k_\alpha \theta$, then:

$$\theta_{\text{ref}} = \frac{\alpha_{\text{cmd}}}{k_\alpha}.$$

If k_α is unknown, assume $k_\alpha \approx 1$ initially and refine it via calibration (see Pre-Lab parameter identification tasks).

5.2.4 Step 4 — Simulink implementation and validation (linear vs. nonlinear)

1. Implement the cascade structure (Figure 5.3):

   - Outer PD: $C_x(s)$ outputs α_{cmd} (or θ_{ref}),
   - Inner PD: $C_m(s)$ tracks θ_{ref} (or α_{ref}),
   - Ball plant: **linear** $P_b(s) = k_b/s^2$ and **nonlinear** $\ddot{x}(t) = k_b \sin(\alpha(t))$ (simulated in parallel).

2. **Implement both models in parallel.** Create two parallel branches after the beam angle signal α:

   - **Linear branch:** use $\ddot{x}(t) = k_b\, \alpha(t)$ (small-angle approximation).
   - **Nonlinear branch:** use $\ddot{x}(t) = k_b\, \sin(\alpha(t))$.

   Both branches should use the same α generated by the inner loop, and both should produce an output $x(t)$.

3. **Compare outputs in the Scope.** Feed the linear and nonlinear ball position outputs into the same `Scope` (two traces). Run the simulation and compare:

   - tracking performance with respect to x_{ref},
   - differences between linear and nonlinear responses (especially under larger angle commands),
   - transient behavior (rise/peak time, overshoot, settling).

   To assist with correct implementation, observe how the model is organized in Fig.5.5

4. Apply step commands in x_{ref} (small displacement, safe range). Record the responses of both models and verify whether the **linear** model remains a good approximation for the operating range used in the laboratory.

5. Verify that the ball position response meets the chosen specifications (using either model, but report both):

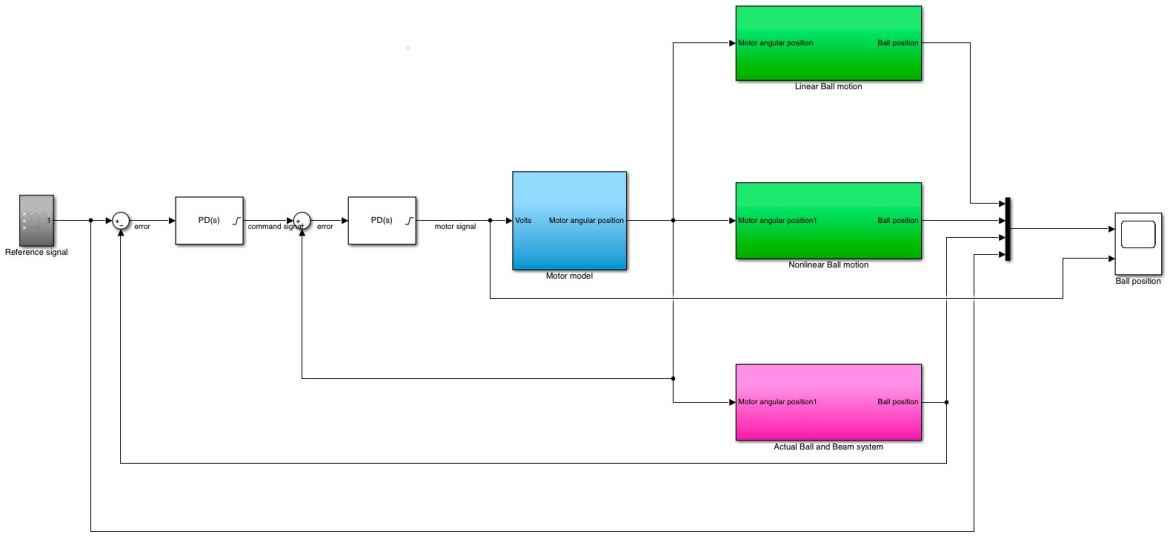


Figure 5.5: Ball and beam cascade control

- Overshoot M_p,
- Peak time t_p,
- Settling behavior and smoothness.

6. If the ball oscillates: increase damping (K_{dx}) or reduce aggressiveness (ω_{nx}).
7. If the response is too slow: increase ω_{nx} (both K_{px} and K_{dx} increase).
8. **Discussion (to include in the report).** State clearly:
   - under which input amplitudes and operating angles the linear and nonlinear outputs become noticeably different,
   - whether the linear model is sufficient for controller design in this lab,
   - how actuator saturation (e.g., limiting α) affects both responses.

6 Lab Report

This chapter provides the complete template and guidelines for writing the lab report associated with the Ball and Beam cascade control experiments. It is structured in four parts: a checklist of mandatory items (Section 6.1), a content template for the ideal PD cascade experiments (Section 6.2), a content template for the practical PD cascade experiments (Section 6.3), and formatting tips (Section 6.4).

6.1 Report checklist for this chapter

In your lab report, include:

A. Controller Design

- The inner-loop specifications and computed gains $K_{p\theta}$, $K_{d\theta}$ using (5.7)–(5.8), with all intermediate steps shown.
- The outer-loop specifications and computed gains K_{px}, K_{dx} using (5.14)–(5.15), with all intermediate steps shown.

B. Simulation Results

- Plots showing the three key system configurations:
  - Open loop (Figure 5.1),
  - Inner-loop closed loop (Figure 5.2),
  - Full cascade closed loop (Figure 5.3).
- All plots must include titles, axis labels with units, legends distinguishing linear/nonlinear/experimental traces, and gridlines.

C. Discussion

- A discussion of differences between the linear and nonlinear ball models when larger angle commands are used.
- A statement on the beam-angle range within which the linear approximation remains valid.
- An explanation of how actuator saturation affects both model responses.

6.2 Template for Content (Cascade Control with Ideal PD Experiments)

6.2.1 Introduction

This section should provide a brief contextualisation of the Ball and Beam system as a benchmark control problem. Students should describe, in their own words, why the open-loop system is

inherently unstable and why a closed-loop strategy is required. The motivation for adopting a cascade structure (inner loop for beam angle, outer loop for ball position) should be clearly stated, linking it to the physical cascade nature of the dynamics derived in Chapter 3.

6.2.2 System Model Summary

Students should summarise the two key transfer functions obtained in Chapter 3.1, identifying the motor/actuator plant $P_m(s)$ and the ball position plant $P_b(s)$. The combined open-loop transfer function should be stated and its order and number of integrators identified. Students should explain, in physical terms, what each integrator represents and why the presence of multiple integrators makes the system challenging to control without feedback.

6.2.3 Inner-Loop Controller Design

Step 1 — Time-Domain Specifications

Students should justify the inner-loop time-domain specifications they have chosen (overshoot M_p and peak time t_p), explaining why the inner loop must be designed to be significantly faster than the outer loop. The chosen values should be within the recommended ranges stated in Section 5.1.2, and the rationale for any specific choice should be discussed.

From the chosen M_p, the damping ratio ζ_θ should be obtained using equation (5.2), showing all intermediate steps. From the chosen t_p, the natural frequency $\omega_{n\theta}$ should be derived using equation (5.3). Students should fill in Table 6.1 with their chosen targets and computed values.

Table 6.1: Inner-loop time-domain specifications.

Parameter	**Symbol**	**Target**	**Computed value**
Overshoot	M_p	$\leq 10\,\%$	
Peak time	t_p	0.2–0.5 s	
Damping ratio	ζ_θ	—	
Natural freq.	$\omega_{n\theta}$	—	

Step 2 — PD Gain Computation

Using the computed ζ_θ and $\omega_{n\theta}$, students should apply equations (5.7)–(5.8) to obtain the proportional and derivative gains $K_{p\theta}$ and $K_{d\theta}$. All intermediate algebraic steps must be shown explicitly. Substituting the system parameters J, B, and K given in Chapter 5, the numerical values of the gains should be reported with at least three significant figures.

Step 3 — Characteristic Polynomial Verification

Students should substitute their computed gains into the closed-loop characteristic polynomial (equation (5.5)) and verify that the resulting poles match the desired locations $s_{1,2} = -\zeta_\theta\,\omega_{n\theta} \pm j\,\omega_{n\theta}\sqrt{1-\zeta_\theta^2}$. This verification demonstrates that the pole-placement procedure was carried out correctly.

In addition, students should plot these poles in the complex plane (s-plane), clearly indicating the real and imaginary axes. They should comment on how the values of ζ_θ and $\omega_{n\theta}$ determine the position of the poles and the expected transient response (e.g., damping and oscillation frequency).

6.2.4 Inner-Loop Simulation Results

Students should describe the step response of the inner closed-loop obtained from Simulink, reporting the measured values of overshoot M_p, peak time t_p, settling time t_s, and steady-state error. These values should be compared against the design targets in Table 6.2, and any discrepancy between the design targets and the simulation results should be explained, noting whether the linear model behaviour is consistent with the pole-placement design.

Students should also include plots of the step response of the inner closed-loop system, clearly showing key features such as overshoot, peak time, and settling time. The figures should have properly labeled axes, units, and legends where appropriate, and should be used to support the discussion of the measured performance metrics and their comparison with the design targets.

Students should also comment on why the inner-loop plant $P_m(s)$, being a type-1 system, produces zero steady-state error for a step reference under PD control, without requiring integral action.

Table 6.2: Inner-loop performance metrics.

Metric	**Target**	**Sim. (linear)**	**Met?**
Overshoot M_p	$\leq 10\,\%$		
Peak time t_p	0.2–0.5 s		
Settling time t_s	—		
Steady-state error	0		

6.2.5 Outer-Loop Controller Design

Step 1 — Time-Domain Specifications

Students should justify the outer-loop specifications, verifying that the chosen outer-loop bandwidth is at least 3–5× lower than the inner-loop bandwidth so that the cascade separation principle applies. The chosen values of M_p and t_p should fall within the ranges stated in Section 5.2.2. Students should fill in Table 6.3.

Table 6.3: Outer-loop time-domain specifications.

Parameter	**Symbol**	**Target**	**Computed value**
Overshoot	M_p	$\leq 5\,\%$	
Peak time	t_p	1.5–3 s	
Damping ratio	ζ_x	—	
Natural freq.	ω_{nx}	—	

Step 2 — PD Gain Computation

Using the computed ζ_x and ω_{nx}, students should apply equations (5.14)–(5.15) to obtain K_{px} and K_{dx}. All intermediate steps must be shown and the numerical values reported.

Step 3 — Characteristic Polynomial Verification

Students should substitute their gains into equation (5.12) and verify that the closed-loop poles of the outer loop match the desired locations $s_{1,2} = -\zeta_x \omega_{nx} \pm j\,\omega_{nx}\sqrt{1-\zeta_x^2}$.

In addition, students should plot these poles in the complex plane (s-plane), clearly indicating the real and imaginary axes. They should comment on how the values of ζ_x and ω_{nx} determine the position of the poles and the expected transient response (e.g., damping and oscillation frequency).

6.2.6 Full Cascade Simulation Results

Students should describe the closed-loop ball position response obtained from the full cascade Simulink model, comparing the linear and nonlinear ball dynamics branches. The measured performance metrics should be reported in Table 6.4 and compared against the design targets.

Students should pay particular attention to any differences between the linear and nonlinear responses, noting under which step amplitudes and beam angle ranges the two models begin to diverge noticeably.

Table 6.4: Full cascade performance metrics — ideal PD.

Metric	**Target**	**Linear model**	**Nonlinear model**	**Met?**
Overshoot M_p	$\leq 5\,\%$			
Peak time t_p	1.5–3 s			
Settling time t_s	—			
Steady-state error	0			

6.2.7 Discussion

Students should address the following points in their discussion:

1. **Linear vs. nonlinear validity.** Over what range of beam angles do the linear and nonlinear ball models produce similar results? Students should identify the approximate angle threshold beyond which the small-angle approximation $\sin\alpha \approx \alpha$ introduces a non-negligible error, and discuss what this implies for the validity of the controller design.

2. **Open-loop instability.** Students should explain analytically, using the poles of the open-loop transfer function $P(s)$, why the system is unstable in open loop and why any constant beam tilt leads to unbounded ball displacement.

3. **Cascade separation.** Students should justify why the inner bandwidth must be significantly higher than the outer bandwidth, and explain under what condition the approximation $\alpha(t) \approx \alpha_{\text{cmd}}(t)$ holds.

4. **Actuator saturation.** Students should discuss how limiting the maximum commanded beam angle affects the transient response of both the linear and nonlinear models, and what consequences excessive angle commands have on the real hardware.

5. **Steady-state error.** Students should explain, based on the system type of each subsystem, why pure PD control achieves zero steady-state error for step references in both loops without requiring integral action.

6.3 Template for Content (Cascade Control with Practical PD Experiments)

6.3.1 Overview and Objectives

This section guides students through the final step of the laboratory: transferring the cascade PD controller designed in Section 6.2 to the real Ball and Beam platform and evaluating how closely the physical system matches the simulation predictions. No additional controller design is required. The provided graphical interfaces handle all communication with the hardware: the MATLAB/SIMULINK interface used for the experiments reported in this section, and the browser-based interface of Section 2.5, which may be used for a preliminary exploration of the gain space before the formal experiment. The student's task is to:

1. Enter the controller gains computed in Sections 6.2.3 and 6.2.5 into the graphical interface.
2. Run the same reference experiments performed in simulation.
3. Record and compare the three key signals — ball position $x(t)$, beam angle $\alpha(t)$, and control signal $\alpha_{\text{cmd}}(t)$ — from both the hardware and the Simulink model.

6.3.2 Experimental Results

Students must record the following performance metrics from the hardware experiment and compare them against the simulation predictions in Table 6.5. The simulation columns should be copied from Tables 6.2 and 6.4.

Table 6.5: Cascade control performance: simulation vs. hardware.

Metric	**Design target**	**Sim. (linear)**	**Sim. (nonlinear)**	**Hardware**
Overshoot M_p	$\leq 5\,\%$			
Peak time t_p	1.5–3 s			
Settling time t_s	—			
Steady-state error	0			

Students should also include plots of the hardware step response showing all three signals. The figures must have properly labelled axes with units and legends, and should be used to support the analysis in Section 6.3.3.

6.3.3 Comparison: Hardware vs. Simulation

The core deliverable of this section is a systematic, figure-based comparison of the three signals recorded from the hardware against the corresponding Simulink traces. All figures must be produced in MATLAB after importing both data sets.

Required figures

1. **Ball position** $x(t)$: one plot with three traces — hardware (solid black), linear simulation (dashed blue), and nonlinear simulation (dashed red) — together with the reference x_{ref} as a dotted grey line. Annotate M_p and t_p directly on the plot.

2. **Beam angle** $\alpha(t)$: same three-trace format. Include horizontal dashed lines to indicate the saturation limits.

3. **Control signal** $\alpha_{\text{cmd}}(t)$: hardware vs. nonlinear simulation (two traces). This plot reveals the effect of real sensor noise on the control effort, which is absent in simulation.

All plots must include titles, axis labels with units, legends, and gridlines, following the formatting guidelines of Section 6.4.

Points to address in the written comparison

1. **Model fidelity.** Which of the two simulation models (linear or nonlinear) better predicts the hardware response? Quantify the difference in M_p and t_p between the best simulation and the hardware measurement. Is the controller designed in simulation directly deployable on the real platform without retuning?

2. **Sources of model–reality mismatch.** Identify and explain the main physical sources of discrepancy between simulation and hardware. Candidates include: unmodelled servo dynamics and dead-band, rolling friction of the ball, beam flexure, sensor latency introduced by the communication, and the discrete-time nature of the controller. Rank these sources qualitatively from most to least significant based on your observations.

3. **Steady-state behaviour.** Does the ball settle exactly at x_{ref} in the hardware experiment? If a non-zero steady-state error is observed, explain its likely cause in terms of the system model (e.g. static friction, servo offset). Note that the simulation predicts zero steady-state error: explain why, based on the system type of each loop.

6.3.4 Discussion

Students should address the following points in their discussion:

1. **Overall agreement between simulation and hardware.** Is the controller designed purely in simulation sufficiently accurate for direct deployment on the real platform? Discuss under what conditions iterative retuning on hardware would be necessary.

2. **Linear vs. nonlinear validity on hardware.** Based on the comparison of both simulation traces against the hardware data, state over what range of step amplitudes and beam angles the linear model is a sufficient basis for controller design. Under what operating conditions would a nonlinear control strategy be necessary?

3. **Actuator saturation.** Discuss how the beam-angle limit affects the transient response on the real hardware and how this compares with the saturation behaviour observed in simulation.

4. **Reproducibility.** Repeat the step experiment at least twice and comment on the repeatability of the hardware response. Identify any differences between runs (e.g. due to ball placement, initial conditions, or servo hysteresis) and discuss their implications for controller robustness.

6.4 Tips for Report Format

6.4.1 Recommended Structure

A well-structured control lab report should contain the sections listed below. Total length: **8–12 pages** including figures and tables.

Table 6.6: Recommended report structure.

Section	Length	Key content
Abstract	1 paragraph	Problem, method, main results from simulation and hardware (5–8 sentences).
Introduction	0.5–1 page	Context, Ball & Beam purpose, cascade motivation.
System Model	0.5–1 page	Equations, parameter values, transfer functions.
Controller Design	1–2 pages	Specs, gain calculations, char. polynomials, pole plots; Tables 6.1 and 6.3 filled in.
Simulation Results	2–3 pages	Figures with captions, linear vs. nonlinear comparison; Tables 6.2 and 6.4 filled in.
Hardware Results	1–2 pages	Hardware traces overlaid with simulation; Table 6.5 filled in.
Discussion	1–2 pages	Model–reality mismatch, sensor noise, saturation, steady-state error, reproducibility.
Conclusions	0.5 page	Summary of findings, limitations, future work.
References	0.5 page	IEEE format recommended.

6.4.2 Guidelines for Figures

- Every figure must have a descriptive caption below it that identifies each trace (model, colour, line style).

- Use a **consistent colour scheme** throughout: one colour per model/condition, identical across all plots.
- Axes must be labelled with quantity name and SI unit, e.g., *Ball position x [m]*, *Time t [s]*.
- Include **gridlines** in all time-domain plots.
- Minimum font size inside figures: 10 pt. Avoid tiny legends.
- Export Simulink Scope plots as PDF or high-resolution PNG ($\geq$ 150 dpi). Avoid raw screenshots at screen resolution.
- **Annotate** key response metrics directly on the plot (e.g., an arrow indicating the peak with a label $M_p = 4.2\,\%$).

6.4.3 Equation and Notation Conventions

- Number all equations that are referenced in the text.
- Use the same variable names as in this workbook ($K_{p\theta}$, $K_{d\theta}$, $\omega_{n\theta}$, ζ_θ, ...) to facilitate cross-referencing.
- Show all intermediate computational steps when deriving gains. A final answer without derivation earns partial credit only.
- Clearly indicate which equations are taken from the workbook and which were derived independently.

6.4.4 Guidelines for Tables

- Each table must have a caption.
- Report numerical values with 3–4 significant figures for gains and 2–3 significant figures for time constants and frequencies.
- Include units in column headers, not in individual cells.

6.4.5 Common Mistakes to Avoid

1. Reporting only the linear model and ignoring the nonlinear comparison.
2. Stating gains without showing how they were computed from specifications.
3. Omitting axis labels or units on plots.
4. Using Simulink block-diagram screenshots instead of actual scope traces as result figures.
5. Practical experiments ignoring the comparison with simulation.
6. Confusing inner-loop and outer-loop specifications or gain equations.

6.4.6 Indicative Grading Rubric

Table 6.7: Grading rubric for the lab report.

Item	**Points**	**Criteria for full credit**
Inner-loop design	20	Correct specs, intermediate steps, final gains.
Outer-loop design	20	Correct specs, intermediate steps, final gains.
Open-loop plot	10	Correct, labelled, instability discussed.
Inner-loop plot	10	M_p and t_p annotated, spec compliance shown.
Cascade plot (ideal PD)	15	Both models, labelled, spec compliance verified.
Practical PD comparison	10	Result comparison.
Discussion quality	10	Linear/nonlinear, saturation, noise addressed.
Format & presentation	5	Captions, labels, equation numbering, clarity.
Total	**100**	

Reports submitted after the deadline incur a 10-point penalty per day.